IF25-11002 PRAKTIKUM PEMROGRAMAN

Pertemuan 11 Searching

Mencari data di array dengan linear search dan binary search, dan membandingkan banyak langkahnya

Tim Pengajar

Tahun 2026. Sejajar dengan Pertemuan 11 Algoritma Pemrograman: Searching.

ITERA

DIKTISAINTEK BERDAMPAK

Program Studi Teknik Informatika
Fakultas Teknologi Industri
Institut Teknologi Sumatera

SEBELUM MULAI

Dua mata kuliah, satu minggu

Topik minggu ini sama di dua mata kuliah, dengan peran yang berbeda.

Algoritma: Searching

Algoritma pencarian beruntun dan biner, prasyarat data terurut, serta menghitung banyak perbandingan.

Praktikum: Searching

Menulis linear search dan binary search di C++, menelusuri low, high, dan mid, serta menghitung perbandingan.

TARGET HARI INI

Saat pulang nanti, kalian bisa

ITERA

1 **Menerapkan**

Menulis program yang mencari data pada array dengan linear search dan binary search, termasuk mencari semua kemunculan dan menangani data yang tidak ditemukan

Sub-CPMK 11.1, CPMK0607

2 **Menjelaskan**

Menelusuri langkah binary search dengan trace table low, high, dan mid, serta menjelaskan perbedaan banyak perbandingan kedua algoritma

Sub-CPMK 11.2, CPMK0608

Dua kemampuan ini dinilai sama berat di Exit Ticket, tugas, UTS, dan UAS.

REVIEW, 5 MENIT

Ingat minggu lalu

Pertanyaan 1	Pertanyaan 2	Pertanyaan 3
Bagaimana menghitung rata-rata setiap kolom array 2D?	Apa elemen diagonal utama matriks?	Apa hasil `s.find("tik")` untuk `"Praktikum"`?

Jawab di kertas dulu, lalu cocokkan dengan teman sebelah.

ITERA

Bagian 1

Masalah pembuka

Mencari nama di daftar wisuda

MASALAH MINGGU INI

Mencari nama di daftar wisuda

Panitia wisuda memegang daftar 1.200 nama yang sudah urut abjad. Seorang orang tua bertanya, apakah anaknya ada di daftar?

Membaca dari nama pertama sampai ketemu bisa butuh ratusan kali melihat. Padahal saat mencari kata di kamus, kita tidak membaca dari halaman pertama.

DISKUSI 2 MENIT

Bagaimana kalian mencari kata di kamus? Mengapa cara itu tidak bisa dipakai pada daftar yang belum urut?

Keluaran

```
Cari: Siti Rahma
Ditemukan di nomor 1023
Linear search : 1023 perbandingan
Binary search : 10 perbandingan
```

SAMBUNGAN DENGAN ALGORITMA

Dari langkah ke instruksi

ITERA

1 Bandingkan satu per satu

Cara paling sederhana: periksa setiap elemen dari depan.

2 Manfaatkan urutan

Bila data urut, buang separuh daftar di setiap langkah.

3 Terjemahkan

Linear search dengan `for`, binary search dengan `while` dan tiga indeks.

Langkah 1 dan 2 dilatih di Algoritma. Langkah 3 kita kerjakan hari ini dengan C++.

ITERA

Bagian 2

Coba dulu, baru dijelaskan

25 menit eksperimen di GDB Online

STRUGGLE, 25 MENIT

Misi kalian

ITERA

1 Cari

Diberikan `int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};`, tampilkan indeks angka 7.

2 Semua

Tampilkan semua indeks angka 9, bukan hanya yang pertama.

3 Tidak ada

Apa yang ditampilkan program kalian bila mencari angka 6?

Boleh

Bertanya ke teman, mengubah apa saja, sengaja membuat error.

Belum dulu

Mencari jawaban jadi di internet atau AI.

DEBRIEF STRUGGLE

Apa yang kalian temukan?

Pertanyaan 1	Pertanyaan 2	Pertanyaan 3
Bagaimana program tahu sebuah data tidak ditemukan?	Kapan `break` dipakai dan kapan tidak?	Berapa perbandingan untuk mencari elemen terakhir?

Jawaban kalian akan kita uji di bagian berikutnya.

ITERA

Bagian 3

Mencari dengan cerdas

Linear search, binary search, dan menghitung langkah

Peta materi hari ini

Langkah	Yang kita bahas	Pertanyaan yang dijawab
1	Linear search	Bagaimana mencari satu data di dalam array?
2	Mencari semua kemunculan	Bagaimana menemukan semua kemunculan, bukan hanya yang pertama?
3	Ide binary search	Bagaimana mencari lebih cepat bila data sudah terurut?
4	Binary search di C++	Bagaimana menulis binary search di C++?
5	Berapa langkahnya?	Seberapa besar beda kecepatan kedua cara mencari?

LANGKAH 1 DARI 5: PERIKSA SATU PER SATU

Linear search

Linear search memeriksa elemen satu per satu dari depan. Begitu cocok, posisinya dicatat dan pencarian berhenti dengan `break`. Bila sampai akhir tidak ada yang cocok, variabel posisi tetap -1.

Linear search bekerja pada data apa pun, terurut atau tidak. Kelemahannya, pada kasus terburuk semua elemen diperiksa.

> **CONTOH SEHARI-HARI**
>
> Mencari kunci motor di tas yang berantakan: kalian mengeluarkan barang satu per satu sampai kuncinya ketemu, atau sampai tas kosong.

Bentuk umum

```
int posisi = -1;
for (int i = 0; i < n; i++)
    if (a[i] == x) { posisi = i; break; }
```

Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

LANGKAH 1 DARI 5: PERIKSA SATU PER SATU

Linear search: contoh

linear.cpp

```
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 7, posisi = -1;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        posisi = i;
        break;
    }
}
if (posisi == -1) cout << "tidak ada" << endl;
else cout << "indeks " << posisi << endl;
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Linear search: jawaban

linear.cpp

```cpp
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 7, posisi = -1;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        posisi = i;
        break;
    }
}
if (posisi == -1) cout << "tidak ada" << endl;
else cout << "indeks " << posisi << endl;
```

Keluaran

```
indeks 3
```

Nilai awal -1 menandai belum ditemukan, karena -1 tidak pernah menjadi indeks yang sah.

LANGKAH 2 DARI 5: TANPA BREAK

Mencari semua kemunculan

Kadang yang ditanyakan bukan di mana pertama kali, tetapi di mana saja dan berapa kali. Untuk itu `break` dihilangkan dan sebuah pencacah ditambahkan.

> **CONTOH SEHARI-HARI**
>
> Seperti mencari semua teman bernama Rani di daftar hadir: kita membaca sampai habis dan mencatat setiap yang cocok.

LANGKAH 2 DARI 5: TANPA BREAK

Mencari semua kemunculan: contoh

semua.cpp

```cpp
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 9, banyak = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        cout << "indeks " << i << endl;
        banyak++;
    }
}
cout << "muncul " << banyak << " kali" << endl;
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

LANGKAH 2 DARI 5: TANPA BREAK

Mencari semua kemunculan: jawaban

semua.cpp

```cpp
int a[8] = {4, 9, 2, 7, 5, 9, 1, 8};
int x = 9, banyak = 0;
for (int i = 0; i < 8; i++) {
    if (a[i] == x) {
        cout << "indeks " << i << endl;
        banyak++;
    }
}
cout << "muncul " << banyak << " kali" << endl;
```

Keluaran

```
indeks 1
indeks 5
muncul 2 kali
```

Untuk semua kemunculan, perulangan tidak dihentikan; pencacah mencatat banyaknya.

Ide binary search

Bila data sudah terurut, kita tidak perlu memeriksa dari depan. Periksa elemen tengah. Bila yang dicari lebih kecil, buang separuh kanan; bila lebih besar, buang separuh kiri. Setiap langkah membuang separuh sisa data.

Tiga indeks menjaga batas: `low` dan `high` mengapit bagian yang masih mungkin, `mid` adalah tengahnya. Trace table berikut menunjukkan pencarian 21.

> **CONTOH SEHARI-HARI**
>
> Tebak angka 1 sampai 100. Penebak yang cerdik selalu menebak tengah, lalu membuang separuh kemungkinan setiap kali diberi tahu terlalu besar atau terlalu kecil.

Ide binary search: ringkasan

Cari 21 pada {3, 8, 15, 21, 34, 42, 57, 66, 79, 90}, indeks 0 sampai 9.

Langkah	low	high	mid	a[mid]	Keputusan
1	0	9	4	34	34 > 21, cari di kiri: high = 3
2	0	3	1	8	8 < 21, cari di kanan: low = 2
3	2	3	2	15	15 < 21, low = 3
4	3	3	3	21	ditemukan di indeks 3

> Binary search hanya benar pada data yang sudah terurut.

Binary search di C++

Terjemahan langsung dari trace table. Perhatikan tiga detail yang sering salah: syarat `low <= high`, pergeseran `mid + 1` dan `mid - 1` (bukan `mid`), dan arah pergeseran yang mengikuti hasil perbandingan.

Bentuk umum

```
int low = 0, high = n - 1;
while (low <= high) {
    int mid = (low + high) / 2;
    // bandingkan a[mid] dengan x
}
```

> Bagian yang ditulis miring di komentar diganti sesuai kebutuhan.

LANGKAH 4 DARI 5: LOW, HIGH, MID

Binary search di C++: contoh

ITERA

biner.cpp

```
int a[10] = {3, 8, 15, 21, 34, 42, 57, 66, 79, 90};
int x = 21, low = 0, high = 9, posisi = -1;
while (low <= high) {
    int mid = (low + high) / 2;
    if (a[mid] == x) { posisi = mid; break; }
    else if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
}
cout << "indeks " << posisi << endl;
```

TEBAK DULU

Sebelum dijalankan, tulis keluarannya di kertas.

Binary search di C++: jawaban

biner.cpp

```cpp
int a[10] = {3, 8, 15, 21, 34, 42, 57, 66, 79, 90};
int x = 21, low = 0, high = 9, posisi = -1;
while (low <= high) {
    int mid = (low + high) / 2;
    if (a[mid] == x) { posisi = mid; break; }
    else if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
}
cout << "indeks " << posisi << endl;
```

Keluaran

```
indeks 3
```

Syaratnya `low <= high`. Dengan `<`, elemen terakhir yang tersisa tidak sempat diperiksa.

Berapa langkahnya?

Pada kasus terburuk, linear search memeriksa semua n elemen. Binary search hanya butuh sekitar $\log_2 n$ langkah, karena data terbagi dua di setiap langkah. Selisihnya makin besar ketika datanya makin banyak.

CONTOH SEHARI-HARI

Seperti mencari nama di buku telepon setebal 1.000 halaman: membaca halaman per halaman bisa butuh 1.000 langkah, membuka di tengah lalu membelah dua hanya sekitar 10.

Berapa langkahnya?: ringkasan

Banyak data	Linear, kasus terburuk	Binary, kasus terburuk
10	10	4
100	100	7
1.000	1.000	10
1.000.000	1.000.000	20

> Binary search jauh lebih cepat, tetapi butuh data terurut. Mengurutkan data adalah materi minggu depan.

SEBELUM LATIHAN

Rangkuman materi

Konsep	Yang perlu diingat
Linear search	Nilai awal -1 menandai belum ditemukan, karena -1 tidak pernah menjadi indeks yang sah.
Mencari semua kemunculan	Untuk semua kemunculan, perulangan tidak dihentikan; pencacah mencatat banyaknya.
Ide binary search	Binary search hanya benar pada data yang sudah terurut.
Binary search di C++	Syaratnya `low <= high`.
Berapa langkahnya?	Binary search jauh lebih cepat, tetapi butuh data terurut.

LATIHAN MENJELASKAN, 3 MENIT

Tebak keluarannya

tebak.cpp

```cpp
int a[7] = {2, 5, 8, 12, 16, 23, 38};
int x = 23, low = 0, high = 6, langkah = 0;
while (low <= high) {
    int mid = (low + high) / 2;
    langkah++;
    if (a[mid] == x) break;
    if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
}
cout << langkah << " " << low << " " << high << endl;
```

> Tulis keluarannya di kertas, karakter demi karakter. Jangan dijalankan dulu.

LATIHAN MENJELASKAN

Tebak keluarannya: jawaban

tebak.cpp

```cpp
int a[7] = {2, 5, 8, 12, 16, 23, 38};
int x = 23, low = 0, high = 6, langkah = 0;
while (low <= high) {
    int mid = (low + high) / 2;
    langkah++;
    if (a[mid] == x) break;
    if (a[mid] < x) low = mid + 1;
    else high = mid - 1;
}
cout << langkah << " " << low << " " << high << endl;
```

Keluaran

```
2 4 6
```

mid = 3 (12 < 23, low = 4), lalu mid = 5 (23, ketemu). Dua langkah; saat berhenti low = 4 dan high = 6.

PESAN DIAMBIL LANGSUNG DARI G++

Error yang sering muncul minggu ini

Kesalahan	Contoh	Pesan pertama dari g++
Membandingkan array dengan ==	`if (a == x)`	`error: ISO C++ forbids comparison between pointer and integer`
Variabel di luar jangkauan	`mid dipakai di luar while`	`error: 'mid' was not declared in this scope`
Kutip tunggal untuk string	`nama[i] == 'Rani'`	`error: no match for 'operator=='`
Posisi tidak dipakai	`int posisi = -1; tak dipakai`	`warning: unused variable 'posisi'`
Kurang titik koma di dalam if	`posisi = mid break;`	`error: expected ';' before 'break'`

Pesan di tabel ini dihasilkan dengan g++ dan opsi -Wall, sama seperti di cek.sh.

Bagian 4

Hands-on bertingkat

45 menit, dari dasar sampai tantangan

Peta hands-on

No	File	Tingkat	Waktu	Yang dilatih
1	`handson1-cari.cpp`	Dasar	10'	linear search, hasil -1, hitung perbandingan
2	`handson2-nama.cpp`	Menengah	15'	mencari semua kemunculan string
3	`handson3-biner.cpp`	Tantangan	20'	trace low-high-mid, perbaiki tiga kesalahan
+	`bonus-banding.cpp`	Pengayaan	sisa	menghitung perbandingan dua algoritma

Kerjakan berurutan. Cocokkan keluaran dengan folder `expected/`; latihan yang membaca masukan memakai data di folder `input/`.

HANDS-ON

Cara mengecek dan aturannya

ITERA

Di GDB Online

Salin isi file latihan, lengkapi TODO, klik Run. Bila program membaca masukan, ketik isi file di `input/`. Bandingkan dengan `expected/`.

Di komputer sendiri

Jalankan `bash cek.sh`. Skrip mengompilasi dengan `-Wall -Werror`, memberi masukan dari `input/`, lalu mencocokkan keluaran.

> AI boleh untuk memahami pesan error, tidak untuk meminta solusi utuh. Exit Ticket dikerjakan tanpa bantuan apa pun.

Bagian 5

Exit Ticket dan tugas

25 menit terakhir, lalu pekerjaan rumah

INDIVIDUAL, 25 MENIT, TANPA AI DAN INTERNET

Exit Ticket 11

Soal	Isi	Poin	CPMK
A1	Tulis linear search yang menampilkan semua indeks kemunculan sebuah nilai	25	CPMK0607
A2	Tulis binary search pada array terurut yang menampilkan indeks atau tidak ditemukan	25	CPMK0607
B1	Isi trace table low, high, mid untuk sebuah pencarian	20	CPMK0608
B2	Jelaskan mengapa binary search gagal pada data yang belum terurut	20	CPMK0608
B3	Bandingkan banyak perbandingan dua algoritma untuk data yang diberikan	10	CPMK0608

Soal A dinilai dari keluaran yang tepat sama. Soal B dinilai dari ketepatan dan alasan.

DIKUMPULKAN SEBELUM PERTEMUAN 12

Tugas 11: Direktori Kontak

ITERA

Bagian A, program, 50 poin

Buat program direktori kontak dengan menu berikut. Data dimasukkan dalam urutan abjad nama; program menolak nama yang tidak urut. Ketentuannya di slide berikut.

Bagian B, penjelasan, 50 poin

Komentar maksimal 150 kata: trace table binary search untuk satu nama yang ada dan satu yang tidak ada, serta perbandingan banyak langkah linear dan binary search pada data kalian. Jelaskan mengapa data harus urut abjad.

Ketentuan Tugas 11

No	Ketentuan Bagian A
1	Tambah kontak, maksimal 10 (nama satu kata dan nomor telepon), dengan validasi urutan abjad
2	Tampilkan semua kontak
3	Cari nama dengan linear search; tampilkan nomor dan banyak perbandingan
4	Cari nama dengan binary search; tampilkan nomor dan banyak perbandingan
5	Keluar
Bonus	Tambahkan pencarian awalan nama, misalnya semua kontak yang diawali "Ra".

EMPAT LEVEL, SAMA DI SEMUA INSTRUMEN

Rubrik Tugas 11

ITERA

Kriteria	Poin	Sangat baik 85–100	Baik 70–84	Cukup 55–69	Kurang <55
A1 Program berjalan	20	tanpa error dan peringatan	ada peringatan	perlu satu perbaikan	tidak terkompilasi
A2 Kelengkapan menu	15	lima menu dan validasi urutan	satu menu kurang	dua kurang	tanpa binary search
A3 Ketepatan pencarian	15	kedua pencarian tepat, termasuk tidak ditemukan	satu kasus salah	dua salah	pencarian salah
B1 Trace table dan perbandingan	30	dua trace tepat dan analisis jelas	satu trace keliru	tanpa analisis	tidak ada atau disalin
B2 Cerita error dan pengujian	20	pesan, sebab, perbaikan, uji	pesan tidak dikutip	hanya perbaikan	tidak ada

Nilai kriteria = poin × skor level. Bagian A total 50, Bagian B total 50.

PERSIAPAN

Sebelum Pertemuan 12

ITERA

1 Selesaikan

Hands-on yang belum lulus `cek.sh`, terutama latihan tantangan.

2 Kumpulkan

Tugas 11 sebelum Pertemuan 12 dimulai.

3 Baca

Modul Belajar 11 untuk mengulang, lalu bagian awal modul berikutnya.

Pertemuan 12 membahas sorting: bubble sort dan selection sort, supaya data bisa diurutkan sebelum dicari dengan binary search.

Terima kasih

Sampai jumpa di Pertemuan 12: sorting